

DS HOMEWORK 4

1) Implement a Stack using Queue data structure.

CODE:

```
#include <stdio.h>
#define MAX 100
int q1[MAX], q2[MAX];
int front1 = 0, rear1 = -1;
int front2 = 0, rear2 = -1;
void push(int x)
{
    int i;
    q2[++rear2] = x;
    while (front1 <= rear1)
        q2[++rear2] = q1[front1++];
    front1 = 0;
    rear1 = -1;
    for (i = front2; i <= rear2; i++)
        q1[++rear1] = q2[i];
    front2 = 0;
    rear2 = -1;
}
void pop()
{
    if (front1 > rear1)
    {
        printf("Stack is empty\n");
        return;
    }
    printf("Popped element = %d\n", q1[front1++]);
}
void display()
{
    int i;
    if (front1 > rear1)
    {
        printf("Stack is empty\n");
        return;
    }
    printf("Stack: ");
    for (i = front1; i <= rear1; i++)
        printf("%d ", q1[i]);
    printf("\n");
}
int main()
{
    int choice, x;
    while (1)
```

```

{
    printf("\n1. Push");
    printf("\n2. Pop");
    printf("\n3. Display");
    printf("\n4. Exit");
    printf("\nEnter choice: ");
    scanf("%d", &choice);
    switch (choice)
    {
        case 1:
            printf("Enter element: ");
            scanf("%d", &x);
            push(x);
            break;
        case 2:
            pop();
            break;
        case 3:
            display();
            break;
        case 4:
            return 0;
        default:
            printf("Invalid choice\n");
    }
}
}

```

OUTPUT:

1. Push
2. Pop
3. Display
4. Exit Enter choice: 3 Stack is empty
5. Push
6. Pop
7. Display
8. Exit Enter choice: 1 Enter element: 45
9. Push
10. Pop
11. Display
12. Exit Enter choice: 3 Stack: 45
13. Push
14. Pop
15. Display

16. Exit Enter choice: 4

2)Implement a Queue using Stack data structure.

CODE:

```
#include <stdio.h>
#define MAX 100
int s1[MAX], s2[MAX];
int top1 = -1, top2 = -1;
void enqueue(int x)
{
    s1[++top1] = x;
}
void dequeue()
{
    int x;
    if (top1 == -1 && top2 == -1)
    {
        printf("Queue is empty\n");
        return;
    }
    if (top2 == -1)
    {
        while (top1 != -1)
            s2[++top2] = s1[top1--];
    }
    x = s2[top2--];
    printf("Deleted element = %d\n", x);
}
void display()
{
    int i;
    if (top1 == -1 && top2 == -1)
    {
        printf("Queue is empty\n");
        return;
    }
    printf("Queue: ");
    for (i = top2; i >= 0; i--)
        printf("%d ", s2[i]);
    for (i = 0; i <= top1; i++)
        printf("%d ", s1[i]);
    printf("\n");
}
int main()
```

```

{
    int choice, x;
    while (1)
    {
        printf("\n1.Enqueue 2.Dequeue 3.Display 4.Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
            case 1:
                printf("Enter element: ");
                scanf("%d", &x);
                enqueue(x);
                break;
            case 2:
                dequeue();
                break;
            case 3:
                display();
                break;
            case 4:
                return 0;
            default:
                printf("Invalid choice\n");
        }
    }
}

```

OUTPUT:

1.Enqueue 2.Dequeue 3.Display 4.Exit

Enter choice: 1 Enter element: 12

1.Enqueue 2.Dequeue 3.Display 4.Exit

Enter choice: 4

3)Write a C program using Stack data structure to implement managing the history of web pages visited.

CODE:

```

#include <stdio.h>
#include <string.h>

```

```

#define MAX 50
char stack[MAX][100];
int top = -1;
void visit(char page[])
{
    if (top == MAX - 1)
        printf("History is full\n");
    else
    {
        strcpy(stack[++top], page);
        printf("Visited: %s\n", page);
    }
}
void back()
{
    if (top <= 0)
        printf("No previous page\n");
    else
    {
        top--;
        printf("Current page: %s\n", stack[top]);
    }
}
void display()
{
    int i;
    printf("\nBrowser History:\n");
    for (i = top; i >= 0; i--)
        printf("%s\n", stack[i]);
}
int main()
{
    int choice;
    char page[100];
    visit("Google");
    while (1)
    {
        printf("\n1.Visit  2.Back  3.Display  4.Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
            case 1:
                printf("Enter webpage: ");
                scanf("%s", page);
                visit(page);
                break;

```

```

        case 2:
            back();
            break;
        case 3:
            display();
            break;
        case 4:
            return 0;
        default:
            printf("Invalid choice\n");
    }
}
}

```

OUTPUT:

1.Visit 2.Back 3.Display 4.Exit

Enter choice: 1

Enter webpage: Insta

Visited: Insta

1.Visit 2.Back 3.Display 4.Exit

Enter choice: 4

4)Write a C program to sort a stack using temporary stack. (implement using stack)

CODE:

```

#include <stdio.h>
#define MAX 100
int stack[MAX], temp[MAX];
int top = -1, ttop = -1;
void push(int x)
{
    stack[++top] = x;
}
void sortStack()
{
    int x;
    while (top != -1)

```

```

    {
        x = stack[top--];
        while (ttop != -1 && temp[ttop] > x)
            stack[++top] = temp[ttop--];
        temp[++ttop] = x;
    }
    while (ttop != -1)
        stack[++top] = temp[ttop--];
}
void display()
{
    int i;
    for (i = top; i >= 0; i--)
        printf("%d ", stack[i]);
    printf("\n");
}
int main()
{
    int n, i, x;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements: ");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &x);
        push(x);
    }
    sortStack();
    printf("Sorted Stack: ");
    display();
    return 0;
}

```

OUTPUT:

Enter number of elements: 2

Enter elements: 1

2

Sorted Stack: 1 2

5)The Stock Span Problem is a financial problem where we have a series of daily price quotes for a stock denoted by an array and the task is to calculate the span of the stock's price

for all days. The span of a stock's price for i^{th} day gives the maximum number of consecutive days leading up to i^{th} day (including current day) where stock's price is less than or equal to its price on day i .

eg:

- I/p: [100, 80, 60, 70, 60, 75, 85]
- O/p: [1, 1, 1, 2, 1, 4, 6]

CODE:

```
#include <stdio.h>
#define MAX 100
int main()
{
    int price[MAX], span[MAX], stack[MAX];
    int n, top = -1;
    int i;
    printf("Enter number of days: ");
    scanf("%d", &n);
    printf("Enter stock prices: ");
    for (i = 0; i < n; i++)
        scanf("%d", &price[i]);
    for (i = 0; i < n; i++)
    {
        while (top != -1 && price[stack[top]] <= price[i])
            top--;
        if (top == -1)
            span[i] = i + 1;
        else
            span[i] = i - stack[top];

        stack[++top] = i;
    }
    printf("Stock Span: ");
    for (i = 0; i < n; i++)
        printf("%d ", span[i]);
    return 0;
}
```

OUTPUT:

Enter number of days: 3

Enter stock prices: 23

67

89

Stock Span: 1 2 3

6) Given an array, for each element find the nearest element to its right that has a higher frequency than the current element. If no such element exists, set value to -1. Implement using stack.

eg:

- I/p: [2, 1, 1, 3, 2, 1]
- O/p: [1, -1, -1, 2, 1, -1]

CODE:

```
#include <stdio.h>
#define MAX 100
int main()
{
    int a[MAX], freq[MAX] = {0};
    int stack[MAX];
    int result[MAX];
    int n, top = -1;
    int i, j;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            if (a[i] == a[j])
                freq[i]++;
        }
    }
    for (i = n - 1; i >= 0; i--)
    {
        while (top != -1 && freq[stack[top]] <= freq[i])
            top--;
        if (top == -1)
            result[i] = -1;
        else
```

```

        result[i] = a[stack[top]];
        stack[++top] = i;
    }
    printf("Output: ");
    for (i = 0; i < n; i++)
        printf("%d ", result[i]);
    return 0;
}

```

OUTPUT:

Enter number of elements: 3

Enter elements: 1

2

3

Output: -1 -1 -1

7) Given a pattern containing only 'I's and 'D's, 'I' for Increasing and 'D' for Decreasing. Device an algorithm using stack to print minimum number following that pattern. Digits from 1-9 and digits can't repeat.

eg:

- I/p: D O/p: 21
- I/p: I D I D I O/p: 121435

CODE:

```

#include <stdio.h>
#include <string.h>
int main()
{
    char str[100];
    int stack[100];
    int top = -1;
    int num = 1;
    int i, n;
    printf("Enter pattern: ");
    scanf("%s", str);
    n = strlen(str);
    printf("Output: ");
    for (i = 0; i <= n; i++)

```

```

{
    stack[++top] = num++;
    if (i == n || str[i] == 'I')
    {
        while (top != -1)
            printf("%d", stack[top--]);
    }
}
return 0;
}

```

OUTPUT:

Enter pattern: DIDIID

Output: 2143576

8) Given an array of digits (values are from 0 to 9), find the minimum possible sum of two numbers formed from digits of the array. All digits of given array must be used to form the two numbers. Use Queue for implementation.

eg:

- **I/p:** [6, 8, 4, 5, 2, 3]
- **O/p:** 604 (min sum formed by numbers 358 and 246)

CODE:

```

#include <stdio.h>
#define MAX 100
struct Queue
{
    int data[MAX];
    int front, rear;
};
void enqueue(struct Queue *q, int x)
{
    q->data[++q->rear] = x;
}
int dequeue(struct Queue *q)
{
    return q->data[q->front++];
}
int main()
{

```

```

struct Queue q;
int a[MAX];
int n, i, j, temp;
int num1 = 0, num2 = 0;
q.front = 0;
q.rear = -1;
printf("Enter number of digits: ");
scanf("%d", &n);
printf("Enter digits: ");
for (i = 0; i < n; i++)
    scanf("%d", &a[i]);
for (i = 0; i < n - 1; i++)
{
    for (j = i + 1; j < n; j++)
    {
        if (a[i] > a[j])
        {
            temp = a[i];
            a[i] = a[j];
            a[j] = temp;
        }
    }
}
for (i = 0; i < n; i++)
    enqueue(&q, a[i]);
for (i = 0; i < n; i++)
{
    if (i % 2 == 0)
        num1 = num1 * 10 + dequeue(&q);
    else
        num2 = num2 * 10 + dequeue(&q);
}
printf("Minimum Sum = %d", num1 + num2);
return 0;
}

```

OUTPUT:

Enter number of digits: 3

Enter digits: 1

2

3

Minimum Sum = 15

9) Given an array and a positive integer k , find the first negative integer for each window (contiguous subarray) of size k . If a window does not contain a negative integer, then print 0 for that window. Implement using queue.

eg:

- **I/p:** [-8, 2, 3, -6, 1], $k = 2$
- **O/p:** [-8, 0, -6, -6]
- First negative integer for each window of size 2:
 - [-8, 2] = -8
 - [2, 3] = 0 (no negative)
 - [3, -6] = -6
 - [-8, 1] = -6

CODE:

```
#include <stdio.h>
#define MAX 100
int main()
{
    int a[MAX], queue[MAX];
    int n, k;
    int front = 0, rear = -1;
    int i;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    printf("Enter window size: ");
    scanf("%d", &k);
    printf("Output: ");
    for (i = 0; i < n; i++)
    {
        if (a[i] < 0)
            queue[++rear] = i;
        while (front <= rear && queue[front] <= i - k)
            front++;
        if (i >= k - 1)
        {
            if (front <= rear)
                printf("%d ", a[queue[front]]);
            else
                printf("0 ");
        }
    }
}
```

```

    }
}
return 0;
}

```

OUTPUT:

Enter number of elements: 5

Enter elements: -8

2

3

-6

1

Enter window size: 2

Output: -8 0 -6 -6

10) Given a matrix of dimension $m \times n$ where each cell in the matrix can have value 0, 1, or 2 which has the following meaning:

- 0: empty cell
- 1: cells having fresh oranges
- 2: cells having rotten oranges

The task is to find the minimum time required so that all oranges become rotten. A rotten orange at index (i, j) can rot other fresh oranges which are its neighbours (up, down, left and right). If impossible to rot every orange, then return -1. Use Queue for implementation.

- eg:
 - I/p: $\begin{bmatrix} 2 & 1 & 0 & 2 & 1 \\ 1 & 0 & 1 & 2 & 1 \\ 1 & 0 & 0 & 2 & 1 \end{bmatrix}$
 - O/p: 2

CODE:

```

#include <stdio.h>
#define MAX 100
struct Queue
{

```

```

    int row[MAX * MAX];
    int col[MAX * MAX];
    int front, rear;
};

void enqueue(struct Queue *q, int r, int c)
{
    q->rear++;
    q->row[q->rear] = r;
    q->col[q->rear] = c;
}

void dequeue(struct Queue *q, int *r, int *c)
{
    *r = q->row[q->front];
    *c = q->col[q->front];
    q->front++;
}

int main()
{
    int a[MAX][MAX];
    struct Queue q;
    int m, n;
    int i, j;
    int r, c;
    int time = 0;
    int fresh = 0;
    int dr[] = {-1, 1, 0, 0};
    int dc[] = {0, 0, -1, 1};
    q.front = 0;
    q.rear = -1;
    printf("Enter rows and columns: ");
    scanf("%d %d", &m, &n);
    printf("Enter matrix:\n");
    for (i = 0; i < m; i++)
    {
        for (j = 0; j < n; j++)
        {
            scanf("%d", &a[i][j]);
            if (a[i][j] == 2)
                enqueue(&q, i, j);
            if (a[i][j] == 1)
                fresh++;
        }
    }
    while (q.front <= q.rear && fresh > 0)
    {
        int size = q.rear - q.front + 1;
        for (i = 0; i < size; i++)

```

```

    {
        dequeue(&q, &r, &c);
        for (j = 0; j < 4; j++)
        {
            int nr = r + dr[j];
            int nc = c + dc[j];
            if (nr >= 0 && nr < m &&
                nc >= 0 && nc < n &&
                a[nr][nc] == 1)
            {
                a[nr][nc] = 2;
                fresh--;
                enqueue(&q, nr, nc);
            }
        }
        time++;
    }
    if (fresh == 0)
        printf("Minimum time = %d", time);
    else
        printf("All oranges cannot be rotten");

    return 0;
}

```

OUTPUT:

Enter rows and columns: 2

2

Enter matrix:

1

2

3

4

Minimum time = 1
